

A Cost-Benefit Study of Template Metaprogramming

1st Miguel Vezanzones Parellada

Technical University of Munich

Munich, Germany

miguel.vezanzones@tum.de

Abstract—Modern high-performance computing applications rely on efficient abstractions to balance performance and usability. This work investigates how increasing compiler visibility through compile-time information affects the generated code and execution efficiency of numerical kernels. In particular, we use template metaprogramming and constexpr evaluation to progressively specialize implementations of the same algorithm by moving information from runtime to compile time.

The impact of specialization is evaluated by comparing tensor contraction kernel with various degrees of specialization. The evaluation considers execution time, compilation overhead, generated code characteristics, code size, and the extent to which compile-time information enables compiler optimizations such as constant propagation. The results illustrate the trade-off between abstraction, generality, and the ability of the compiler to generate efficient machine code.

Index Terms—template metaprogramming, C++, code generation, compile-time specialization, constexpr evaluation

I. INTRODUCTION

Modern processors are increasingly complex and utilizing them effectively requires close collaboration with the compiler. The code must expose sufficient information to the compiler so that optimizations can be applied. However, high-level abstractions often hide structural properties of a program until runtime, limiting the transformations available during compilation.

C++ aims to provide a simple and direct mapping to hardware, and zero-cost abstractions [1, p. 2] as its core philosophy. One of the main design goals of the zero-overhead principle is that the cost of using high-level abstractions should be zero, as compared to its hand-crafted counterpart [2].

Template metaprogramming allows extending this principle to reducing the cost of abstractions beyond what a programmer could produce by hand. This is possible by allowing and guiding the compiler into generating extremely thin layers of abstraction whose complexity is resolved at compile time.

Compiler visibility into the source and knowledge of values and types determines the transformations that the compiler can perform, and the constructs that it can be directed into producing.

Compile time values are accessible to the compiler in constexpr contexts, which can be used in code generation (embedding literals into assembly immediate arguments¹).

¹For example: `add eax, 1`. Here the integral literal 1 has been embedded in source, while `eax` contains a runtime value stored in a register

Similarly, templated types allow the compiler to compute types at compile time [1, p. 6]. Template metaprogramming techniques exploit this mechanism to compute arbitrary types and guide code generation. Template metaprogramming is used in highly-optimized applications to afford efficient abstractions [3].

Compile time information must be present in the source for the compiler to be able to use it. This applies most often to values, but the same thing applies to types. Compile time information can be lowered to runtime information in a nested context, reducing the compiler’s visibility moving forward within that nested context. This process is irreversible, and defines a flow of information from its source to its destination, where it is eventually consumed.

Compile time information can be particularly useful in specific applications such as numerical kernels, where the structure of the computation often depends on properties such as dimensions, data layout, and iteration patterns. Tensor contraction is selected as the reference application in this work because it exposes several independent dimensions of compile-time specialization. Tensor rank, tensor dimensions, and contraction indices can either be represented dynamically or encoded directly into the generated implementation, allowing controlled comparison between different levels of compiler visibility.

Please find the source code of this project at `constexpr_propagation`.²

II. COMPILE-TIME INFORMATION

This section describes how compile-time information can affect code generation to understand the potential benefits that we aim to quantify in this work.

A. Compile-Time Work

It is often desirable to perform computation at compile time. Often we aim to compute values such that they do not need to be recomputed at runtime; computing types is also often desirable for type safety and efficient type specialization [1, p. 6].

²https://github.com/MiguelVezanzones/constexpr_propagation

1) *Move computation from runtime to compile time:* Moving any computation from runtime to compile time should increase compile times in favor of runtime. The compiler executes C++ in an internal interpreter, so it is significantly more limited than the runtime [4].

2) *Avoid redundant computation at runtime:* Often, efficient implementations expect work to be done by the caller, such as an output parameter in matrix multiplication. This often happens because work performed behind an API call must be done per call, not per caller. Moving this computation to the compile time can enable efficient and clean APIs.

3) *Compile-Time Complexity:* Cost of recompilation increases as the compiler is responsible for more work. Template metaprogramming techniques can become a burden if recompilation is frequent. This often happens with heavily templated Boost libraries.

Modern C++ standards have improved on this aspect by providing explicit language support, with constructs such as `if constexpr` and `template for`.

B. Code Generation

Code generation is influenced significantly by compiler visibility and the flexibility required for the code.

1) *Enable optimizations:* Hard-coding information about the size of a loop can enable optimizations such as automatic loop unrolling and auto-vectorization.

2) *Code Size:* Code size is affected in various ways by template metaprogramming, most often code grows due to template duplication. However, templated code can often be simpler and smaller once compiled.

3) *Type Simplification:* The use of templates and compile-time values can allow using types with stronger invariants, such as replacing a `std::vector` with a `std::array`. Properties that are often desirable are `triviality`, `standard layout`, `trivial copyability`.

4) *Compile-time contracts:* Compile-time computation can be also used to enforce contracts at compile time, which is a strong invariant that the runtime cannot provide.

C. Architectural

1) *Reduce memory operations:* Assembly immediate arguments bypass the normal load path, and are rather fetched by the frontend along with their instruction. This reduces the overall number of load operations in the assembly. What type categories and bit widths are allowed is architecture-dependent. Note that Intel 64 (Intel's version of x86-64) supports small integral and no floating point immediates, while AMD64 does also support floating point immediates to some extent.

2) *Reduce function arguments:* Compile-time information or uncertainty determines what regions of the memory model can be used. The heap is the least restrictive and can be used at runtime, but with higher costs of allocation. Stack memory is fast, but the compiler must know the size of objects, and their lifetime is context-determined. The most restrictive section is the static/globals, which have fixed locations in the binary.

These can be constants or read/write values. The static/global constants (located in `.rdata`) are the most powerful for the compiler.

3) *Reduce register pressure:* Immediate arguments reduce register pressure as they never become part of the in-flight or architectural state, but are rather consumed and discarded.

4) *Read-only data and coherence protocols:* Read only data regions (`.rdata`) are simpler to manage by a multithreaded architecture as it promises stronger invariants. Race conditions and initialization failures are not an issue for compile-time values.

III. DESIGN SPACE

This project aims to evaluate the tradeoffs of heavy template metaprogramming and `constexpr` evaluation. The design choice we focus on is: runtime versus compile-time parameters. This decision will be evaluated on its impact on flexibility against performance.

IV. COMPUTE KERNEL

The extent to which compile-time information can be exploited effectively is highly application-dependent. Therefore, a reference application must be chosen where the availability of such information has a direct impact on both the algorithmic structure and implementation strategy.

Tensor contraction has been selected as the reference compute kernel due to its relevance in scientific computing workloads, as well as its ability to exploit compile-time information. A tensor contraction generalizes matrix multiplication by combining two tensors over one or more shared dimensions while preserving the remaining dimensions as output dimensions. The operation can be expressed as a summation over a set of contraction indices:

$$C_{i,j} = \sum_k A_{i,k} B_{k,j}$$

Unlike fixed-rank linear algebra operations, tensor contractions require handling additional structural information, including tensor rank, dimension sizes, memory layout, and contraction indices. When this information is only available at runtime, the implementation must rely on dynamic mechanisms such as runtime shape traversal, and generalized iteration schemes with a multi-index. In contrast, when this information is available at compile time, the implementation can specialize the generated kernel by resolving iteration structures and intermediate values during compilation.

This makes tensor contraction a suitable benchmark for evaluating the impact of compile-time information. For this, we consider three variants that are explained below.

V. IMPLEMENTATION

We consider three levels of compile-time information that the implementation can exploit in a meaningful way. Each level will build on top of the previous one, always elevating information from runtime to compile time. These levels are denominated *runtime rank*, *compile-time size* and *compile-time*

sizes (v1, v2, v3 respectively), according to how the available information affects each implementation.

These can be thought of as different specification standards with different requirements for compliance.

A. Runtime Rank (v1)

This level specifies the lowest degree of compile-time information expected by each of the implementations. It requires static typing and static binding. This is a sensible baseline as it is often taken for granted.

Static typing implies that the concrete types being used are known to the compiler (*i.e.*, no `std::any` or `void*`). Static binding implies that function calls are resolved at compile time (*i.e.*, no virtual dispatch or function pointers).

In this context, the compiler has full visibility of the concrete types and call structure, enabling optimization across function boundaries and efficient code generation. However, no algorithm-specific information such as ranks, dimensions or contraction patterns will be accessible during code generation.

1) *v1::tensor (t1)*: This variant must use the heap to store both its data and its metadata. The data buffer will be a contiguous buffer of `T`. Similarly, the metadata buffer will contain the tensor sizes and strides for efficient access with a multi-index. Accessing these buffers through a pointer indirection does not meaningfully affect runtime on its own, as compared to the heap. However, we will have 7 different allocations³ which will demand more memory subsystem resources.

2) *v1::contraction (c1)*: Since the rank of the tensors is unknown at compile time, the first-level contraction implementation uses a multi-index to iterate over the tensors. The multi-index uses an internal `for` loop for each increment, introducing significant overhead in iteration.

B. Compile-Time Rank (v2)

The second level requires that the rank of the tensors and the contraction degree be known at compile time.

1) *v2::tensor (t2)*: Knowing the rank of the tensor allows the strides and sizes of the tensors involved to be stored within the tensor object itself. Thanks to this compile time information, the metadata of the tensor will be stored in the stack, rather than the heap, and accessible without a pointer indirection.⁴ The data itself must still be heap-allocated, however, as the size of each dimension is not known.

2) *v2::contraction (c2)*: Knowing the rank of the tensors significantly simplifies iterating over the tensors⁵. This implementation sets up two sets of nested `for` loops using template metaprogramming, removing the need to iterate using a multi-index.

³Two allocations for each tensor and one for the contraction index set. Each tensor uses two buffers in the heap, these could be merged into one, but the additional complexity was deemed unimportant.

⁴Moving data from the stack to the heap also removes a costly memory allocation, but these are not included in the benchmark loop.

⁵The rank of a tensor corresponds to the number of nested `for` loops required to iterate over it in a structured way.

The internal structure of this iteration scheme is shown in Listing 1. Given a range of sizes of size `Rank`, this function stamps one nested loop for each dimension and keeps track of the indices per dimension. In the inner most loop, a callback is invoked, adding the indices as an additional argument.

```
template <
    std::integral Size_Type,
    Size_Type I,
    Size_Type Rank>
constexpr auto shaped_for_impl(
    std::ranges::sized_range auto const& sizes,
    std::ranges::sized_range auto& idxs,
    auto&& fn,
    auto&&... args
) noexcept -> void
{
    if constexpr (I == Rank)
    {
        std::invoke(
            std::forward<decltype(fn)>(fn),
            std::forward<decltype(args)>(args) ...,
            idxs
        );
        return;
    }

    for (idxs[I] = 0; idxs[I] != sizes[I];
        ++idxs[I])
    {
        shaped_for_impl<Size_Type, I + 1, Rank>(
            sizes,
            idxs,
            std::forward<decltype(fn)>(fn),
            std::forward<decltype(args)>(args) ...
        );
    }
}
```

Listing 1. Nested loop iteration

C. Compile-Time Sizes (v3)

The third level requires that the sizes of the tensors and the indices to contract over, be known at compile time.

1) *v3::tensor (t3)*: This affects the tensor implementation in two major ways. For once, the data in the tensor may be stored in the stack, significantly improving locality, especially for small tensors. Additionally, tensor sizes and strides become part of the type, reducing the size of each instance.

A limit of 128 KB per tensor has been used to conditionally store the data in the heap to prevent stack overflow.

2) *v3::contraction (c3)*: Knowing the size of the tensors allows propagating these values into the loop limits into the two sets of nested `for` loops. This enables inlining the loop limits into the assembly as integral immediate arguments, and enables aggressive loop unrolling and vectorization.

Additionally, iteration functions now require less input arguments, since the location of static data is known at compile time. This change makes feasible computing the target linear index into each tensor during iteration, removing an inner product computation between the strides and the multi-index on each tensor access.⁶

⁶This same optimization was unsuccessful for the *v2::contraction* iteration scheme, presumably for the significant increase in function arguments.

D. Signatures

Listing 2 shows the signatures of the tensor types and the tensor contraction.⁷

```
using T = double;           // value_type
std::size_t N = \* \*;     // Rank
std::size_t Sx = \* \*;    // N Sizes

// Tensor signatures
using t1 = v1::tensor<T>;
using t2 = v2::tensor<T, N>;
using t3 = v3::tensor<T, S1, ..., SN>;

// Tensor contraction singatures
v1::contract(v1::compatible auto const& A,
             v1::compatible auto const& B,
             v1::compatible auto & C,
             v1::index_set auto const& cis);

v2::contract(v2::compatible auto const& A,
             v2::compatible auto const& B,
             v2::compatible auto & C,
             v2::index_set auto const& cis);

v3::contract<CIS>(v3::compatible auto const& A,
                 v3::compatible auto const& B,
                 v3::compatible auto & C);
```

Listing 2. Signatures

To allow generic use of the contraction implementations, template arguments are deduced from the input arguments. Only `v3::contraction` requires an additional Non-Type Template Parameter (NTTP) argument as it is necessary to determine the shape of the output tensor, which must be known at compile time.

E. Compatibility

Contraction and tensor implementations of different levels are compatible because each level extends the previous one with stricter requirements and compile time information can be lowered to runtime information. Since the opposite is not true, compatibility goes in one direction, making `v3::tensor` compatible with any contraction implementation, `v2::tensor` compatible with `v2::contraction` and `v1::contraction`, and `v1::tensor` only compatible with `v1::contraction`.

This will be used to assess what part of the performance improvements can be attributed to changes in the tensor implementation or the contraction implementation.

VI. EXPERIMENTAL SETUP

All benchmarks were conducted on a system with the following specifications shown in Tab. I.

The compiler was configured with aggressive optimization settings to focus benchmark differences on code generation:

```
-O3 -fno-exceptions -march=native -mavx
-fno-math-errno -ffast-math
-fno-trapping-math -flto
```

⁷Names have been simplified.

TABLE I
HARDWARE AND SOFTWARE ENVIRONMENT USED FOR BENCHMARKING.

Operating System	Arch Linux
Kernel	6.18.35-1-lts
Compiler	GCC 16.1.1
Language Standard	C++26
CPU	Intel Core i7-9750H
DRAM	16 GB DDR4-2667 MT/s
Cores / Threads	6 / 12
Base Frequency	2.6 GHz

These optimization flags enable high-level, whole-program compiler optimizations, target the native CPU architecture, enable AVX vector instructions, allow more aggressive floating-point transformations.

VII. RESULTS

Elevating information to the compile time leads to a series of tradeoffs, and must be evaluated with metrics that capture these. We will primarily use the following set of objective

- `runtime`: Main metric to measure code efficiency
- `compilation time`: Measure how compilation times increase with compile time computation and template metaprogramming.
- `code size`: Measure the impact of template metaprogramming in binary size.

Additionally, we will look at the generated assembly to observe the effects of constant propagation and the presence of immediate arguments.

A. Runtime

Most often, compile time information is exploited to improve performance in expenses of flexibility. Runtime is, therefore, the most important metric we will analyze in order to measure potential gains. Whether those gains are justifiable depends on the application and should be carefully considered.

The benchmark consists of 63 contractions of tensors of various sizes and ranks. Sizes range from 2 to 64 per dimension, ranks range from 2 to 6, and contracted indices range from 1 to 4 pairs. Not all combinations are present in the benchmark as exponential growth rapidly exhausts system resources.

Runtime is plotted against the number of floating point operations needed for the naive implementation we are using, which amounts to about two floating point operations per inner-most loop iteration.

For the following plots, C1, C2, C3 stand for the three contraction implementations, and T1, T2, T3 stand for the three tensor implementations.

Runtime has been measured using Google Benchmark, and mean time over 10 repeated runs is reported.⁸

⁸Note that Google Benchmark performs iterations until convergence in each run.

1) `v1::contraction`: Fig. 1 shows how the tensor implementation impacts performance for the same contraction algorithm that does not exploit the specific tensor implementation. The difference lies in the overhead of accessing tensor data and metadata.

The improvement obtained by replacing `v1::tensor` with `v2::tensor` is relatively small. The effects responsible for the improvement should be improved locality for the metadata, which is now stored in the stack along the metadata of the other tensors.

Using `v3::tensor` as the tensor implementation leads to more significant improvements. Now, tensor indexing should be much more efficient, as the strides are compile time constants that the compiler can inline. Additionally, tensor data is stored in the stack for small tensors, which improves performance for small enough tensors significantly.

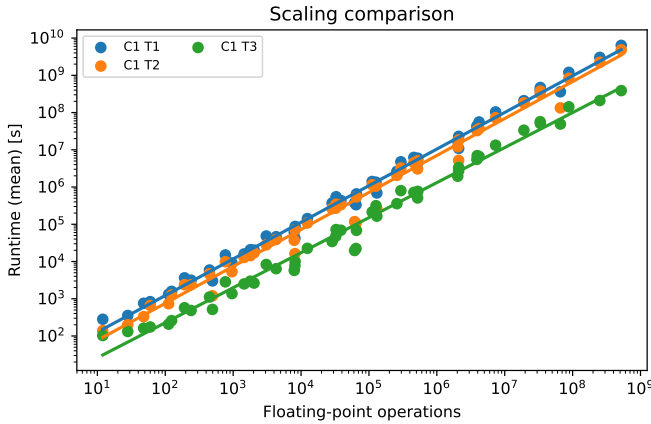


Fig. 1. `v1::contraction` performance scaling with each compatible tensor.

2) `v2::contraction`: This implementation can use the rank of the tensors and contraction index set at compile time, bypassing the need to iterate with multi-indices and removing allocations entirely from the benchmark loop. Here, `v3::tensor` improves upon the baseline with better locality for small sizes and more efficient data access. Fig. 2 shows the results graphically.

For sizes below a few hundred operations, the benchmarking framework is not able to accurately measure runtime and we get essentially constant values. This could be solved by benchmarking natively with a simple stopwatch; however, these examples are pathological and unimportant.

3) `v3::contraction`: This contraction implementation is only compatible with `v3::tensor`, as it expects compile time sizes from the input and output tensors. For completeness, Fig. 3 shows the scaling of this implementation graphically. This implementation propagates constants to the loop control limits

4) *Overview*: Fig. 4 shows the performance of each of the three variants, along with `numpy`'s `tensor_dot` for reference (denoted by "np np").

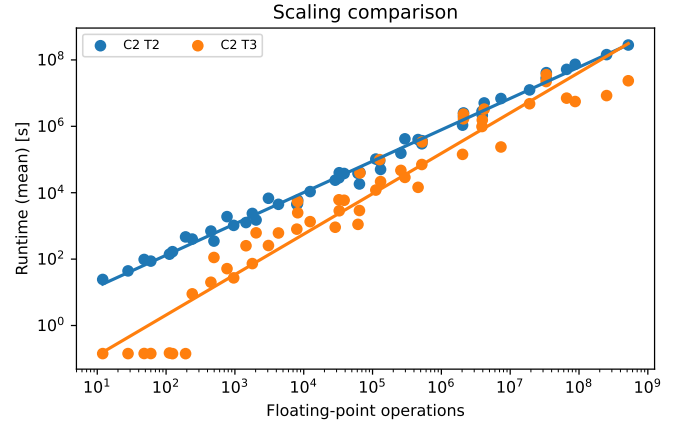


Fig. 2. `v2::contraction` performance scaling with each compatible tensor.

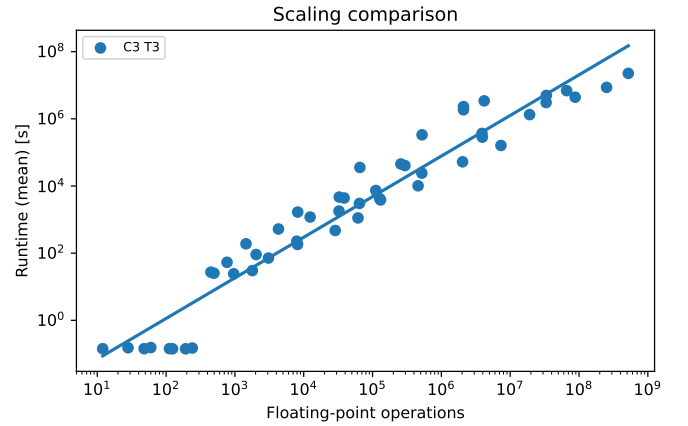


Fig. 3. `v3::contraction` performance scaling.

A direct comparison with `numpy` is not fair, as its implementation is fully generic and most likely respects IEEE 754 floating point arithmetic. However, it serves as an interesting comparison with an optimized library, to show the value of tight implementations in High-Performance Computing (HPC) projects.

`numpy` has a significant overhead from function calls, preprocessing to optimize iteration patterns (such as permuting the order of loops), which is only amortized for small to medium size tensors.

Finally, Fig. 5 gathers all plots for completeness.

B. Compile Time

Template metaprogramming demands the compiler to perform significant type manipulations, and `constexpr` evaluation requires the compiler to execute unoptimized C++ code in an internal interpreter.

Fig. 6 shows that `v3::tensor` introduces a significant compile time overhead, increasing compile times by up to three times in the current examples.

This increase in compile time was expected and not particularly problematic for these small micro benchmarks, but could

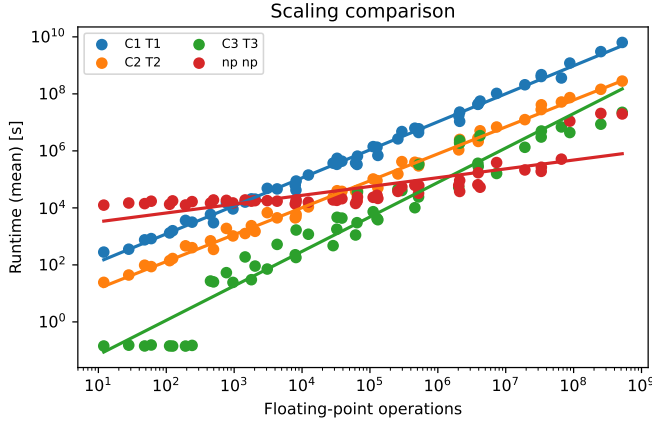


Fig. 4. Example of a figure caption.

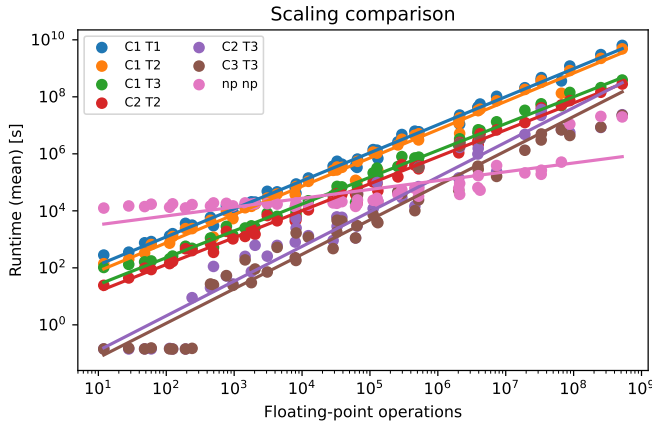


Fig. 5. Example of a figure caption.

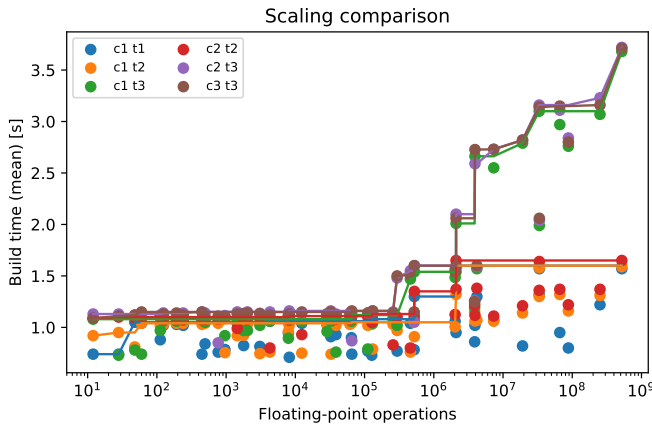


Fig. 6. Example of a figure caption.

become a problem in larger, more sophisticated applications. The real limitation, however, is that compile time computation cannot scale to the same order of magnitude as the runtime. For this reason, compile time computation cannot ever replace runtime computation, it is rather a complement to it, both need to be balanced within the context of an application.

C. Code Generation

In terms of code generation, we will analyze constant propagation into immediate arguments in the `v3` variant and code size.

1) *Assembly immediates*: We only consider this variant because it is the only implementation where the compiler has enough information such that it should not embed it everywhere. This latter point will be illustrated below.

For `v2`, all of the compile time information provided is explicitly exploited, and is, therefore, not up to the implementation to embed in the assembly. An example of this scenario are the nested `for` loops shown in Listing 1; the implementation is not allowed to choose an alternative, as we have explicitly stamped those loops. Similarly, the stack arrays that contain tensor metadata are explicitly instantiated, so the compiler is forced to use the compile time rank specified by the `v2` variant. The compiler is free to optimize the code in multiple ways that would essentially remove these constructs we have explicitly requested, but in such cases the compiler is choosing to transform code that already has these values embedded.

In `v3`, however, we, for example, explicitly propagate `constexpr` values into the limits of `for` loops. If these loop limits were embedded in the assembly, there would need to be an assembly block for each such loop, as each must contain their own loop limits. However, since we are using regular, runtime loops⁹, we are not forcing the compiler into stamping many variations of the same loop, rather, we are providing the resources required to do it. The compiler will then decide whether to inline `constexpr` values (such as the loop limits) in the assembly based on its tradeoffs, primarily runtime performance and code size.

In the generated assembly of `v3` examples, we have observed that loop limits have often been embedded in the assembly as immediate arguments, while tensor strides have not been embedded in similar logic used for linear index calculation. This seems to be because the compiler is actively exploiting a regularity present in most of the benchmark data points. In particular, we mostly used tensors with the same size per dimension, which significantly reduces the times that code duplication would happen by embedding the value (if the same value is always embedded, then no duplication is necessary).

Even these regular tensors have different strides per dimension, which explains why the compiler decided to load the strides from memory at runtime to reuse the same code, instead of stamping variants for every stride size.

⁹As opposed to the compile time recursion loop used in Listing 1 to stamp `for` loops.

2) *Code Size*: No significant source code size differences have been observed across implementations.

VIII. DISCUSSION

// TODO: Sorry I couldnt finish

IX. CONCLUSION

// TODO: Sorry I couldnt finish

REFERENCES

- [1] B. Stroustrup, *Foundations of C++*, Berlin Heidelberg, 2011. [Online]. Available: <https://www.stroustrup.com/ETAPS12-corrected.pdf>
- [2] *Zero-overhead principle - cppreference.com*. Accessed: Jun. 26, 2026. [Online]. Available: https://en.cppreference.com/cpp/language/Zero-overhead_principle
- [3] A. Hendricks, “Lessons Learned: Template-Heavy C++ in Production HPC Runtime Systems,” en, in *Proceedings of the SC '25 Workshops of the International Conference for High Performance Computing, Networking, Storage and Analysis*, St Louis MO USA: ACM, Nov. 2025, pp. 1009–1016, ISBN: 979-8-4007-1871-7. DOI: 10.1145/3731599.3767476 Accessed: May 6, 2026. [Online]. Available: <https://dl.acm.org/doi/10.1145/3731599.3767476>
- [4] *Constant Interpreter — Clang 10 documentation*. Accessed: Jun. 26, 2026. [Online]. Available: <https://releases.llvm.org/10.0.0/tools/clang/docs/ConstantInterpreter.html>